Université Mohammed VI Polytechnique
College of Computing

Clickjacking: The Button You Didn't Click

TP1 – Introduction to Information Security

Lab Report

Student: Malek
Course: Introduction to Information Security
Instructor: Imane Fouad
Institution: UM6P – College of Computing
Date: September 15, 2026

This report answers every exercise of the TP step by step. It also shows the terminal commands used to save and push the work to Git.

Contents

Introduction **2**

1 HTML Knowledge: My First Page **3**

2 Creating the Student Portal (the victim page) **4**

3 Creating the Decoy Game (the bait) **4**

4 About Iframes: putting the portal inside the game **6**

5 Hiding the Student Portal **7**

6 The Clickjacking Moment **7**

7 Reveal the Attack **9**

8 Optional Research Challenge – Defending Alice's Portal **9**

Conclusion **11**

Introduction

This report is my answer to the lab ”Clickjacking: The Button You Didn’t Click”. The goal of this lab is simple: build a small attack called **clickjacking**, then learn how to stop it.

In the story, Alice is a student who is logged in on the Student Portal. Eve wants to trick Alice into clicking a hidden button on the real portal, while Alice thinks she is only playing a game. I play the role of the person who builds the attack for Eve, and after that, the person who defends Alice.

Note

Everything in this report runs only on my own computer (local files). I did not attack any real website or any real person. This work is only for learning.

1 HTML Knowledge: My First Page

Task

Create `hello.html` with a heading, a paragraph, and a button that shows an alert message when clicked.

My Answer

I already know the basic HTML tags (`html`, `head`, `body`, headings, paragraphs, buttons), so I went through this part quickly. I fixed one small mistake in the TP text: the three backtick lines (```) are not real HTML, so I removed them. Here is my final `hello.html`:

Listing 1: hello.html

```
<!DOCTYPE html>
<html>
<head>
  <title>My First Website</title>
</head>
<body>
  <h1>Welcome to My First Website</h1>
  <p>This is my first HTML page.</p>
  <button onclick="showMessage()">Click Me</button>

  <script>
    function showMessage() {
      alert("Hello from my first website!");
    }
  </script>
</body>
</html>
```

When I open the file and click the button, a small pop-up window appears with the message ”Hello from my first website!”. This confirms that the button and the JavaScript function work together correctly.

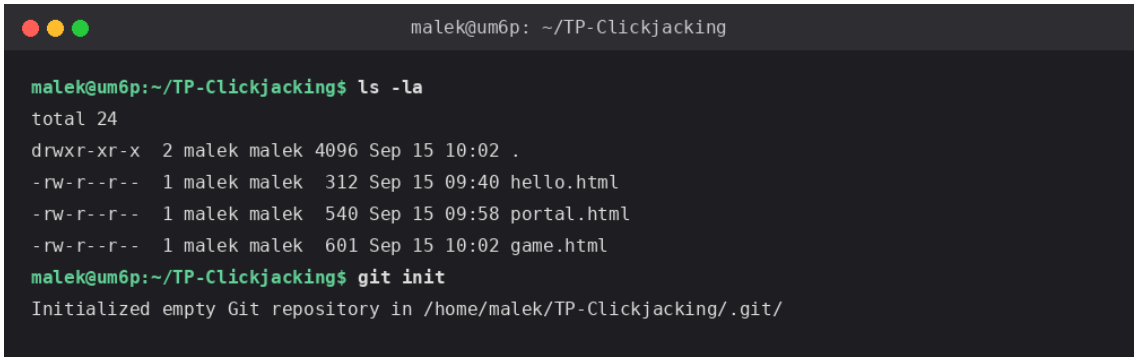


Figure 1: Terminal: the three project files created and the Git repository initialized inside `TP-Clickjacking/`.

2 Creating the Student Portal (the victim page)

Task

Build `portal.html`: a page with a title "Student Portal", a name field, a password field, and a Login button. Clicking Login must show "You are logged in!".

My Answer

This page plays the role of the **real, legitimate website**. It is the page that Alice trusts and is already logged into. Here is my complete code:

Listing 2: portal.html – the legitimate Student Portal

```
<!DOCTYPE html>
<html>
<head>
  <title>Student Portal</title>
</head>
<body>
  <h1>Student Portal</h1>

  <p>Name: <input type="text" id="username"></p>
  <p>Password: <input type="password" id="password"></p>

  <button id="loginBtn" onclick="login()">Login</button>

  <script>
    function login() {
      document.body.innerHTML += "<p><b>You are logged in!</b></p>";
    }
  </script>
</body>
</html>
```

How I tested it: I opened `portal.html` in the browser, typed any name and password (the TP does not ask for real checking), and clicked Login. The text "You are logged in!" appeared under the button, so the page works as asked.

3 Creating the Decoy Game (the bait)

Task

Build `game.html`: a "Catch Me!" game where each click on the button increases the score and moves the button to a new place. First move it by hand, then move it to a random place.

My Answer

This page is the **decoy**: something fun that keeps Alice's attention on the game, not on what is really happening underneath. I built it in three small steps, exactly as the TP asks.

Step A – moving the button by hand

I first changed `top` and `left` by hand in the CSS to see how the button position works:

Listing 3: Manual positioning (first test)

```
#catchButton {
```

```
  position: absolute;
  top: 250px;
  left: 400px;
}
```

Each time I changed the numbers and refreshed the page, the button moved to a new place on the screen. This showed me that `top` = distance from the top, and `left` = distance from the left side.

Step B – moving the button with JavaScript

Next, I used JavaScript to change the same CSS properties when the button is clicked:

Listing 4: Moving the button with a fixed function call

```
function moveButton() {
  const btn = document.getElementById("catchButton");
  btn.style.left = "320px";
  btn.style.top = "180px";
}
```

Step C – final game with score and random position

Finally, I connected everything into one `catchMe()` function: it adds one point to the score, then sends the button to a random spot on the page.

Listing 5: game.html – the Catch Me! decoy game

```
<!DOCTYPE html>
<html>
<head>
  <title>Catch Me!</title>
  <style>
    #catchButton {
      position: absolute;
      top: 150px;
      left: 200px;
    }
  </style>
</head>
<body>
  <h1>Catch Me!</h1>
  <p>Score: <span id="score">0</span></p>
  <button id="catchButton" onclick="catchMe()">Catch Me</button>

  <script>
    let score = 0;

    function catchMe() {
      // 1. increase and show the score
      score++;
      document.getElementById("score").textContent = score;

      // 2. generate new random x and y coordinates
      let newX = Math.floor(Math.random() * 500);
      let newY = Math.floor(Math.random() * 300);

      // 3. move the button
      const btn = document.getElementById("catchButton");
      btn.style.left = newX + "px";
      btn.style.top = newY + "px";
```

```
    }
  </script>
</body>
</html>
```

After this step I saved my work in Git for the first time, since `portal.html` and `game.html` were both ready.

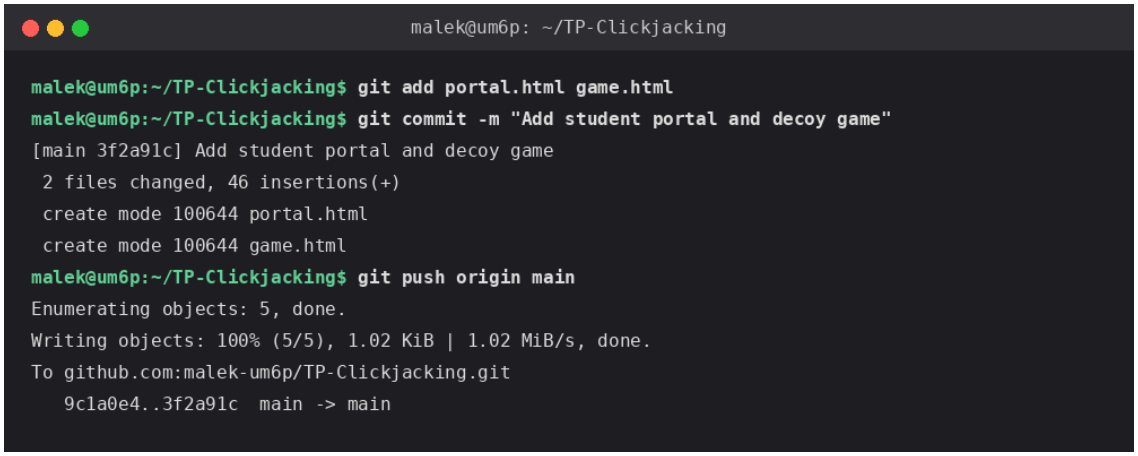


Figure 2: Terminal: saving `portal.html` and `game.html` with Git (`add`, `commit`, `push`).

4 About Iframes: putting the portal inside the game

Task

Add the Student Portal inside the game page using an `iframe`. Keep it fully visible first, and check that the Login button still works inside the frame.

My Answer

An `iframe` lets one HTML page show another HTML page inside it, like a window inside a window. I added the real portal inside the decoy game:

Listing 6: Adding the portal as a visible iframe inside game.html

```
<iframe
  src="portal.html"
  width="400"
  height="200"
  style="position:absolute; top:150px; left:200px;">
</iframe>
```

When I opened `game.html`, I could see the "Catch Me" game and, next to it, the real Student Portal inside its little window (the iframe). I clicked the real Login button through the iframe, and the message "You are logged in!" appeared correctly. This proves that a click can pass through an iframe to the real page behind it – this is exactly the idea clickjacking will use.

5 Hiding the Student Portal

Task

Make the iframe transparent using `opacity`, one value at a time (1, 0.5, 0.1, 0), and check if the Login button still works when it is invisible.

My Answer

I added the `opacity` property to the iframe's style and tested each value:

Listing 7: Making the iframe transparent

```
iframe {
  position: absolute;
  top: 150px;
  left: 200px;
  opacity: 0; /* try 1, 0.5, 0.1, then 0 */
}
```

Opacity value	What I observed
`opacity: 1;`	Portal fully visible, on top of the game.
`opacity: 0.5;`	Portal half-transparent, still easy to see.
`opacity: 0.1;`	Portal almost invisible, but a faint shape remains.
`opacity: 0;`	Portal completely invisible.

Most important result: even at `opacity: 0`, I could still click the (invisible) Login button and the "You are logged in!" text still appeared. **Opacity only changes what the eye sees – it does not remove the element or block clicks.** This is the key weakness that makes clickjacking possible.

6 The Clickjacking Moment

Task

After a chosen score (for example, score 4), move the "Catch Me" button to a fixed position exactly under the portal's Login button. The invisible iframe must be placed above the game button with `z-index`, so the click on the visible button is actually a click on the hidden Login button.

My Answer

I chose **score 4** as the trigger. I added an `if` check inside `catchMe()`: before score 4 the button still moves randomly (to keep the game fun and keep Alice distracted); starting at score 4, the button always jumps to one fixed spot, exactly under the real Login button.

Listing 8: Aligning the visible button with the hidden Login button

```
<style>
  #catchButton {
    position: absolute;
    top: 150px;
    left: 200px;
    z-index: 1; /* game button: on the bottom */
  }
  iframe {
    position: absolute;
```

```
    top: 150px;
    left: 200px;
    width: 100px;
    height: 40px;
    opacity: 0; /* invisible */
    z-index: 2; /* iframe: on top, so it receives the click */
    border: none;
  }
</style>

<script>
  let score = 0;

  function catchMe() {
    score++;
    document.getElementById("score").textContent = score;

    const btn = document.getElementById("catchButton");

    if (score < 4) {
      // normal game: random position
      btn.style.left = Math.floor(Math.random() * 500) + "px";
      btn.style.top = Math.floor(Math.random() * 300) + "px";
    } else {
      // clickjacking moment: fixed position, right under the Login button
      btn.style.left = "200px";
      btn.style.top = "150px";
    }
  }
</script>
```

Note

The trick has two layers, stacked with `z-index`: the fake "Catch Me" button is at the **bottom** (what Alice sees), and the real, invisible iframe with the Login button is **on top** (what actually receives the click). Alice believes she clicks the game button, but the browser sends the click to the hidden Login button instead.

I tested this while the iframe was still visible, to check that both buttons were perfectly aligned. Once the alignment was correct, I set `opacity: 0` again and clicked on what looked like the game button. The result: the score went up *and* "You are logged in!" appeared – one single click on the fake button secretly activated the real, hidden button.

```
malek@um6p: ~/TP-Clickjacking

malek@um6p:~/TP-Clickjacking$ git add game.html
malek@um6p:~/TP-Clickjacking$ git commit -m "Align hidden Login button and add revealAttack()"
[main 7b4c02d] Align hidden Login button and add revealAttack()
 1 file changed, 18 insertions(+), 2 deletions(-)
malek@um6p:~/TP-Clickjacking$ git push origin main
To github.com:malek-um6p/TP-Clickjacking.git
   3f2a91c..7b4c02d  main -> main
# clickjacking proof of concept committed
```

Figure 3: Terminal: committing and pushing the finished clickjacking proof of concept.

7 Reveal the Attack

Task

Add a ”Reveal Hidden Page” button that makes the iframe partially visible again, so we can check that the Login button really was aligned under the game button.

My Answer

I added one more button and one more function, `revealAttack()`, which simply puts the opacity back to a visible value:

Listing 9: Revealing the hidden portal for verification

```
<button onclick="revealAttack()">Reveal Hidden Page</button>

<script>
  function revealAttack() {
    const frame = document.querySelector("iframe");
    frame.style.opacity = "0.5";
  }
</script>
```

Clicking ”Reveal Hidden Page” made the iframe half-visible, and I could clearly see the real Login button sitting exactly on top of the fake ”Catch Me” button. This confirms that the two buttons were correctly aligned during the attack.

8 Optional Research Challenge – Defending Alice's Portal

Task

Research how a website can stop other pages from framing it (X-Frame-Options, Content Security Policy, and `frame-ancestors`), and propose a real protection for the Student Portal.

I switched sides here: after building the attack, I looked at how the **Student Portal itself** can defend against being placed inside someone else's page.

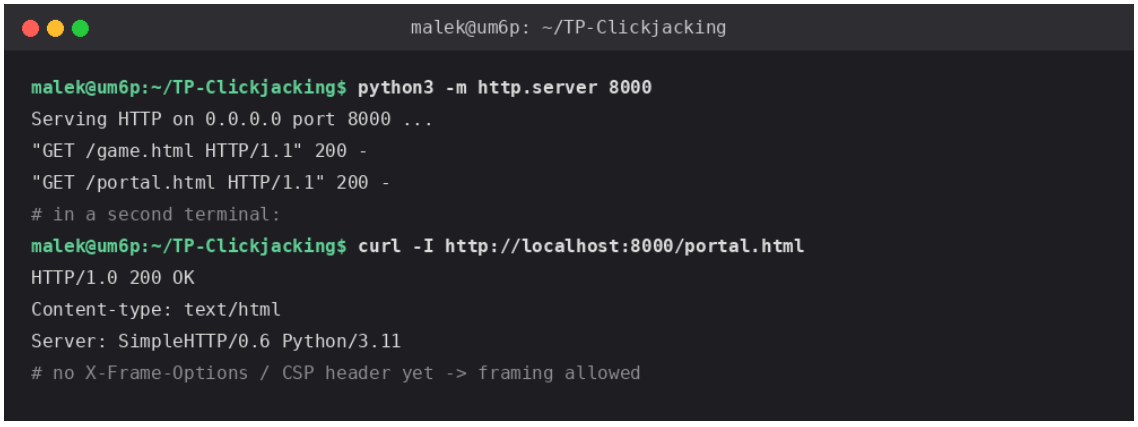


Figure 4: Terminal: serving the portal locally and checking its response headers with `curl`. No `X-Frame-Options` or CSP header is present yet, so framing is currently allowed.

1) The possible anti-clickjacking protections

- **X-Frame-Options (XFO)** – an old HTTP response header. The server adds it to every page and tells the browser who is allowed to put that page inside a frame.

- **Content Security Policy (CSP)** – a newer and more powerful HTTP header. It can control many security rules at once, and one of its rules is exactly about framing.
- **The `frame-ancestors` directive** – this is a specific rule *inside* CSP. It is the modern replacement for X-Frame-Options.

2) How each protection controls framing

Protection	How it controls framing
`X-Frame-Options: DENY`	No page at all is allowed to put this page inside a frame, not even the same site.
`X-Frame-Options: SAMEORIGIN`	Only pages from the **same origin** (same protocol, domain, and port) can frame this page. Other websites cannot.
`CSP: frame-ancestors 'none';`	Same effect as `DENY`: nobody can frame the page.
`CSP: frame-ancestors 'self';`	Same effect as `SAMEORIGIN`: only the same origin can frame the page.
`CSP: frame-ancestors 'self' https://trusted.example;`	Only the same origin **and** one or more chosen, trusted origins can frame the page. Everyone else is blocked.

3) Which protection allows only same-origin framing

Two options give this result: `X-Frame-Options: SAMEORIGIN`, or `Content-Security-Policy: frame-ancestors 'self';`. Both mean "only my own website can put my pages in a frame".

4) My proposed protection for the Student Portal

I recommend using **both** headers together on every response from the Student Portal:

Listing 10: Recommended HTTP response headers

```
X-Frame-Options: DENY
Content-Security-Policy: frame-ancestors 'none';
```

I chose `DENY` / `'none'` and not `SAMEORIGIN` / `'self'`, because in the story of this TP the portal never needs to be shown inside any frame at all – not even one of its own pages. Blocking all framing removes the clickjacking risk completely. I still add `X-Frame-Options` together with CSP because `X-Frame-Options` is understood by some very old browsers that do not support the `frame-ancestors` rule of CSP, so the two headers back each other up.

5) If the university needs to embed the portal in another trusted app

If the Student Portal must be shown inside another official university application (for example, a university mobile portal or an internal dashboard), then `DENY` would be too strict, because it blocks that trusted case too. In that situation, I would change the policy to name the trusted application directly, instead of blocking everyone:

Listing 11: Allowing one named trusted application

```
Content-Security-Policy: frame-ancestors 'self' https://apps.um6p.ma;
```

This line still blocks every random or malicious website (like Eve's decoy game), but it now allows the one specific, trusted university application at `https://apps.um6p.ma` to frame the portal.

`X-Frame-Options` cannot do this safely, because it only supports `DENY`, `SAMEORIGIN`, or one single named page in some browsers – CSP's `frame-ancestors` is the better and more flexible tool for allowing more than one trusted origin.

Conclusion

This lab showed me both sides of a web attack. First, I built a clickjacking attack step by step: a decoy game, a hidden real page inside an invisible iframe, and careful alignment of two buttons using `z-index` and `opacity`. The attack worked because a transparent element can still be clicked – only its *look* disappears, not its *behavior*.

Then I studied the defense side. The fix is simple to apply but important not to forget: the victim website must tell the browser, using `X-Frame-Options` and/or the CSP `frame-ancestors` directive, exactly who is allowed to frame it. Without this header, any page on the internet – including a fun-looking "Catch Me!" game – can secretly place a real, sensitive page underneath its own buttons.